

2026학년도 2학기 언어데이터과학

제9강 문자 인코딩과 유니코드

박수민

서울대학교 인문대학 언어학과

2026년 9월 30일 수요일

지난 시간에 배운 것

- 1 토큰 빈도·타입 빈도·문서 빈도는 서로 다른 개념이다.
- 2 TTR과 STTR로 어휘의 크기를 재고, 지프의 순위-빈도 법칙을 보았다.
- 3 단발어가 생산성의 재료가 되는 것을 확인했다.
- 4 “무엇을 하나로 셀 것인가”를 두 번 더 묻기로 했다.

오늘의 목표

- 1 부호화와 복호화가 무엇인지 설명하고 예를 들 수 있다.
- 2 한글 인코딩의 역사를 알고 유니코드가 왜 필요했는지 말할 수 있다.
- 3 한글 문자를 UTF-8 3바이트로 손으로 부호화할 수 있다.
- 4 코퍼스 파일을 열 때 무엇을 적어야 하는지 안다.

‘바람’의 길이는 2인가 6인가

노트북 입력

```
w = '바람'  
print(len(w))  
print(len(w.encode('utf-8')))
```

실행 결과

```
2  
6
```

둘 다 맞다. 다만 센 것이 다르다.

8강에서 ‘바람’은 토큰 빈도 5, 문서 빈도 1인 낱말이었다. 그 ‘바람’의 **길이**를 물었더니 두 가지 답이 나왔다. 하나는 **글자**를 셸고, 하나는 **바이트**를 셸다.

오늘은 그 둘 사이에 무엇이 있는지 본다.

3강이 남겨 둔 약속

터미널 입력

```
wc -c data/apicius/recipes.txt  
wc -m data/apicius/recipes.txt
```

실행 결과

```
367682 data/apicius/recipes.txt  
367377 data/apicius/recipes.txt
```

3강 슬라이드에 이렇게 적어 두었다

“가’는 1글자이지만 3바이트이다. 왜 그런지는 9강에서 다룬다.”

영어 요리책인데도 **305**만큼 어긋났다.

오늘의 질문

풀 떡밥

- 1 3강: `wc -c`와 `wc -m`의 차 **305**는 무엇인가?
- 2 4강: `[^A-Z]`를 지웠더니 `ENTRÉE`가 `ENTRE`가 된 까닭은?
- 3 8강: ‘바람’의 길이는 2인가 6인가?

오늘도 새 데이터는 없다

3강부터 써 온 요리책과 8강의 동요 10편뿐이다. 도구가 바뀌어도 파일은 그대로다.

셋 다 같은 것을 묻는다. 문자 하나가 기계 안에서 무엇인가?

1 부호화

부호화와 복호화
비트와 바이트

2 아스키와 확장 아스키

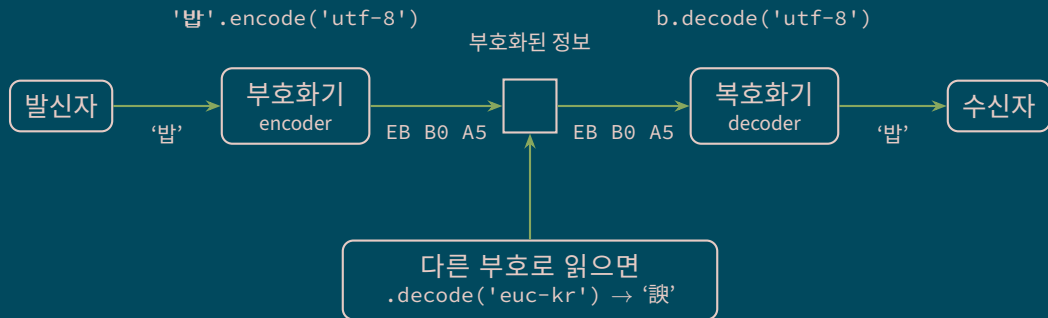
3 한글 인코딩

4 유니코드와 UTF-8

5 코퍼스를 다룰 때

6 마무리

정보는 형태를 바꾸어 건너간다



Shannon. (1948). A Mathematical Theory of Communication. *The Bell System Technical Journal* 27, 379-423. <Figure 1>의 Information Source·Transmitter·Receiver·Destination·Noise Source를 문자 인코딩에 맞추어 다시 그렸다.

부호화한 것을 **같은 부호로** 복호화해야 원래 문자가 돌아온다. 오늘 볼 깨짐은 모두 이 한 줄을 어긴 것이다.

모형의 네 낱말

부호화-복호화 모형의 주요 개념

부호화 정보를 다른 형태로 바꾸는 처리 (encoding)

복호화 부호화된 형태를 원래 정보로 되돌리는 처리 (decoding)

부호 정보를 바꾸는 규칙 체계 (code)

코덱 부호화·복호화를 수행하는 기계나 알고리즘 (codec)

목적

표준화, 보안, 압축 등

부호화 예시

- 알파벳 → 모스 부호
- 집 → 주소
- 음악 → 악보
- 이미지 → 픽셀
- 평문 → 암호문

오늘의 부호: 문자 인코딩

문자 인코딩

문자 인코딩 문자 $\rightarrow$ N 바이트 이진수

UTF-8에서 한글 '가'는 3바이트 **11101010 10110000 10000000**이 된다.

되돌아오는 것이 핵심이다

부호화한 것을 **같은 부호로** 복호화해야 원래 문자가 나온다. 부호화만 잘하는 것은 아무 쓸모가 없다.

영어 쓰는 사람들에게는 한 바이트도 넉넉해서...

1바이트는 256가지 정보를 담는다

이진수	십진수	십육진수
0000	0	0
0001	1	1
0010	2	2
0011	3	3
0100	4	4
0101	5	5
0110	6	6
0111	7	7
1000	8	8
1001	9	9
1010	10	A
1011	11	B
1100	12	C
1101	13	D
1110	14	E
1111	15	F

1바이트

단위 8비트 = 이진수 여덟 자리 = 십육진수 두 자리
가짓수 256 (0과 255 사이의 정수)

십진수 111을 십육진수로

$111_{(10)} = 1101111_{(2)} \rightarrow$ 네 자리씩 끊어 0110 1111
 $\rightarrow 6F_{(16)}$

이진수 네 자리가 십육진수 한 자리다. 그래서 1바이트를 십육진수 두 자리로 적는다. wc -c가 센 367,682는 이 바이트의 개수다.

1 부호화

2 **아스키와 확장 아스키**
아스키 코드
확장 아스키

3 한글 인코딩

4 유니코드와 UTF-8

5 코퍼스를 다룰 때

6 마무리

ASCII는 7비트로 영어를 담았다

ASCII

American Standard Code for
Information Interchange

7비트($2^7 = 128$ 가지)로 영어를 적는
인코딩 방식

128가지의 내역

제어 문자 33 + 공백 1 + 숫자 10 +

영문자 52 + 특수 문자 32 = **128**

2025년 판은 제어 문자 33개를 제외해
95가 되었다.

b7 b6 b5 b4				b3 b2 b1 b0				Column	0	1	2	3	4	5	6	7
Row	0	1	2	3	4	5	6	7	0	1	2	3	4	5	6	7
0	0	0	0	0	0	0	0	0	NUL	DLE	SP	0	@	P	\	p
1	0	0	0	1	1	1	1	1	SOH	DC1	!	1	A	Q	a	q
2	0	0	1	0	2	STX	DC2	"	2	B	R	b	r			
3	0	0	1	1	3	ETX	DC3	#	3	C	S	c	s			
4	0	1	0	0	4	EOT	DC4	\$	4	D	T	d	t			
5	0	1	0	1	5	ENQ	NAK	%	5	E	U	e	u			
6	0	1	1	0	6	ACK	SYN	&	6	F	V	f	v			
7	0	1	1	1	7	BEL	ETB	'	7	G	W	g	w			
8	1	0	0	0	8	BS	CAN	(	8	H	X	h	x			
9	1	0	0	1	9	HT	EM	)	9	I	Y	i	y			
10	1	0	1	0	10	LF	SUB	*	:	J	Z	j	z			
11	1	0	1	1	11	VT	ESC	+	;	K	[	k	{			
12	1	1	0	0	12	FF	FS	,	<	L	\	l				
13	1	1	0	1	13	CR	GS	-	=	M	]	m	}			
14	1	1	1	0	14	SO	RS	.	>	N	^	n	~			
15	1	1	1	1	15	SI	US	/	?	O	_	o	DEL			

https://commons.wikimedia.org/wiki/File:USASCII_code_chart.png

표에서 A를 찾아본다

아스키 코드에서 A에 대응하는 값 찾기

1000001 (가로 0001 → 오른쪽 네 자리, 세로 100 → 왼쪽 세 자리)

0	0	0	0	1	0	0	0	0	0
0	0	0	0	0	NUL	DLE	SP	0	@
0	0	0	1	1	SOH	DC1	!	1	A

의문

8비트를 채워야 1바이트가 되는데, 나머지 한 비트는 어디에 있는가?

남은 한 비트는 검산에 썼다

패리티 비트 (parity bit)를 앞에 붙인다

... ..

@ 1000000 → **1**1000000

A 1000001 → **0**1000001

B 1000010 → **0**1000010

C 1000011 → **1**1000011

... ..

패리티 비트의 값 정하기

0 1의 개수가 짝수인 경우

1 1의 개수가 홀수인 경우

11101100처럼 값이 맞지 않으면 전송 중에 오류가 생겼음을 알 수 있다.

지금은

오류 검사가 통신 계층으로 옮겨 가면서 여덟째 비트가 비었다. 그 빈자리를 노린 것이 다음 절이다.

빈 한 비트에 유럽 문자를 넣었다

확장 아스키 코드

8비트($2^8 = 256$ 가지)를 모두 써서 한 문자를 한 바이트로 적는 방식

- 기존 아스키 코드의 문자
- 확장 로마자 (é, ç, ñ 등)
- 기호 (×, ÷, ±, ¼)

지역별 예시

ISO 8859-1 서유럽

ISO 8859-2 동유럽

ISO 8859-3 남유럽

ISO 8859-4 북유럽

ISO 8859-5 키릴 문자

... ..

<https://www.ecma-international.org/publications-and-standards/standards/ecma-94/>

ISO 8859-1(서유럽)

				b ₀	0	0	0	0	0	0	0	0	1	1	1	1	1	1		
				b ₁	0	0	0	0	1	1	1	1	0	0	0	0	1	1		
				b ₂	0	0	1	1	0	1	1	0	0	1	1	0	0	0		
				b ₃	0	1	0	1	0	1	0	1	0	1	0	1	0	1		
				00 01 02 03 04 05 06 07 08 09 10 11 12 13																
b ₄	b ₃	b ₂	b ₁																	
0	0	0	0	00			SP	0	à	P	`	p				NBSP	°	À	Đ	
0	0	0	1	01			!	1	A	Q	a	q					ì	±	Á	Ñ

바이트만 보아서 는 무슨 글자인지 알 수 없다. 어느 포로 읽을지를 함께 정해야 한다.

ISO 8859-2(동유럽)

				b ₀	0	0	0	0	0	0	0	0	1	1	1	1	1	1	
				b ₁	0	0	0	0	1	1	1	0	0	0	0	1	1	0	
				b ₂	0	0	1	1	0	1	1	1	0	0	1	1	0	0	
				b ₃	0	1	0	1	0	1	0	1	0	1	0	1	0	1	
				00 01 02 03 04 05 06 07 08 09 10 11 12 13															
b ₄	b ₃	b ₂	b ₁																
0	0	0	0	00			SP	0	à	P	`	p				NBSP	°	Ř	Đ
0	0	0	1	01			!	1	A	Q	a	q				À	à	Á	á

ISO 8859-2

파이썬으로 확인한다

노트북 입력

```
from unicodedata import lookup

n1 = lookup('LATIN CAPITAL LETTER N WITH TILDE')
n2 = lookup('LATIN CAPITAL LETTER N WITH ACUTE')
print(n1, n1.encode('ISO 8859-1'))
print(n2, n2.encode('ISO 8859-2'))
print(b'\xd1'.decode('ISO 8859-1'), b'\xd1'.decode('ISO 8859-2'))
```

실행 결과

```
Ñ b'\xd1'
Ó b'\xd1'
Ñ Ó
```

305는 우리 요리책 안에 있었다

노트북 입력

```
from collections import Counter

data = open('data/apicius/recipes.txt', 'rb').read()
text = data.decode('utf-8')
odd = [c for c in text if not c.isascii()]
print(len(data), len(text), len(odd), len(set(odd)))
print(Counter(odd).most_common(5))
```

실행 결과

```
367682 367377 305 20
[('æ', 116), ('Æ', 70), ('é', 38), ('À', 17), ('É', 16)]
```

$367,682 - 367,377 = \mathbf{305}.$

4강에서 사라진 é가 이 목록에 있다

4강에서 본 것

[^A-Z]로 알파벳 아닌 것을 모두 지웠더니 ENTRÉE가 ENTRE가 되었다. é가 A-Z 범위에 없었기 때문이다.

오늘 붙이는 이름

é는 ISO 8859-1의 **c9**번 자리에 있는 글자다. 아스키 코드 00-7F 밖이라 A-Z로는 잡히지 않는다.

두 표는 겹치지 않는다

한 표에는 256칸뿐이다. 유럽 글자를 넣으면 한글 자리가 없고, 한글을 넣으면 유럽 글자 자리가 없다. 실제로 'é'.encode('euc-kr')는 오류를 낸다.

1바이트로는 세상의 글자를 다 담을 수 없다. 한글은 어떻게 했을까?

1 부호화

2 아스키와 확장 아스키

3 한글 인코딩

2바이트의 벽

완성형

확장 완성형

글자가 깨지는 까닭

4 유니코드와 UTF-8

5 코퍼스를 다룰 때

6 마무리

한글은 한 바이트에 들어가지 않는다

한글 인코딩의 문제

현대 한글(옛한글 제외) 음절 글자 수는 **11,172**개다.

- $2^8 = 256$ 보다 많고 $2^{16} = 65,536$ 보다 적다.
- 1바이트로는 표현할 수 없다. 최소한 2바이트가 필요하다.

11,172라는 수는 어디에서 왔는가

초성 19가지 × 중성 21가지 × 종성 28가지 = 11,172
(종성 28가지에는 “받침 없음”이 들어 있다.)

이 세 값들은 뒤에서 다시 나온다. 기억해 두자.

한글 인코딩에는 역사가 있다

한글 인코딩 방법

역사 N 바이트 조합형, 3바이트, 7비트 완성형, 2바이트 조합형, 2바이트 완성형, 확장 완성형, ...

표준 유니코드 (UTF-8, UTF-16, UTF-32)

두 갈래

조합형 초성·중성·종성에 각각 번호를 주고 조립한다

완성형 완성된 글자마다 번호를 하나씩 준다

한동안 완성형이 이겼다. 그리고 문제가 시작되었다.

2,350자만 고른 표

KS C 5601-1987 (지금의 KS X 1001)

한계

한글 2,350자만 표현할 수 있다.

- 똘방각하 (×)
- 뽕시콜라 (×)
- 설미 (×)
- 똥어 (×)
- 뽕, 켁, 똘, ... (×)



천계영. 《엔플러그드 보이 2》. 서울문화사. 1997.

이름을 못 쓰는 사람이 생긴다

‘김설의’라는 이름이 입력되지 않는 이유는 은행이나 통신사, 대학 등 민간에서 사용하는 대형 전산시스템의 한국산업표준(KS)이 ‘한글조합형코드’가 아닌 ‘한글완성형코드’(EUC-KR)인 탓이다. 완성형코드는 국제표준과 충돌이 적다는 장점이 있지만, 미리 조합되어 있는 글자 외의 문자는 인식할 수 없다는 단점이 있다. ‘의’, ‘꺾’ 같이 빈도수가 낮은 문자들은 코드에 등록하지 않아 문자로 보지 못하는 셈이다. 이 완성형코드는 한글 초·중·종성으로 조합 가능한 한글 문자 1만1172자 중 2350자만 표현할 수 있다.

http://www.hani.co.kr/arti/society/society_general/864914.html

인코딩은 기술이면서 정책이다

표에서 빠진 글자는 **쓸 수 없는 글자**가 된다. 누구의 이름을 적을 수 있는지가 표 하나로 정해진다.

‘뭇’과 ‘미’ 사이에 ‘믹’가 없다

노트북 입력

```
for ch in ['뭇', '믹', '미']:  
    print(ch, ch.encode('euc-kr'))
```

실행 결과

```
뭇 b'\xb9\xcb'  
믹 b'\xa4\xd4\xa4\xb1\xa4\xd2\xa4\xd4'  
미 b'\xb9\xcc'
```

2바이트가 아니라 8바이트다

앞뒤 글자는 CB → CC로 붙어 있는데 그 사이에 있어야 할 ‘믹’만 **8바이트**다.

8바이트의 정체는 조합형이다

바이트 쌍	뜻
A4 D4	채움 문자
A4 B1	ㅁ
A4 D2	ㄴ
A4 D4	채움 문자

노트북 입력

```
print(b'\xa4\xb1'.decode('euc-kr'))
print(b'\xa4\xd2'.decode('euc-kr'))
```

실행 결과

```
ㅁ
ㄴ
```

표에 없는 글자는 자모를 늘어놓아 흉내 냈다

채움 문자(U+3164 HANGUL FILLER)는 “여기는 비었다”는 뜻이다. 초성 자리와 종성 자리가 비었으니 ㅁ + ㄴ를 늘어놓은 것이다. 한 글자에 8바이트를 쓴다.

통합 완성형은 빠진 글자를 2바이트에 넣었다

Unified Hangul Code, 코드 페이지 949

의의

2,350자 밖의 글자에도 2바이트 코드를 배당했다. 이제 11,172자를 모두 쓸 수 있다.

대가

새로 넣은 글자가 **가나다순 자리에 들어가지 못했다**. 2,350자 바깥의 남은 자리에 밀어 넣었기 때문이다.

‘**의**’가 생겼다. 그런데 B9 CB과 B9 CC 사이가 아니라 92 DE다.

노트북 입력

```
for ch in ['뫣', '의', '미']:  
    print(ch, ch.encode('cp949'))
```

실행 결과

```
뫣 b'\xb9\xcb'  
의 b'\x92\xde'  
미 b'\xb9\xcc'
```

남은 문제 (1): 코드 순으로 줄을 세우면 가나다순이 아니다

노트북 입력

```
w = ['미', '의', '못']  
print(sorted(  
    w,  
    key=lambda c: c.encode('cp949')))
```

실행 결과

```
['의', '못', '미']
```

정렬 결과에 주목

CP949의 코드 순이 가나다순과
어긋난다.

바람직한 한글 인코딩의 요건

- 1 현대 한글 11,172자를 모두 담을 것
- 2 자모순으로 배열할 것

→ 통합 완성형은 1번만 지켰다.

남은 문제 (2): 둘째 바이트가 영문자와 겹치기도 한다

노트북 입력

```
b = '똥'.encode('cp949')
print(b)
print(hex(b[0]), hex(b[1]), chr(b[1]))
```

실행 결과

```
b'\x8cc'
0x8c 0x63 c
```

b'\x8cc'의 끝 글자가 왜 c로 보이는가

‘똥’의 둘째 바이트가 63, 곧 아스키 소문자 c와 같은 값이다. 그래서 바이트 단위로 자르거나 grep하면 **없는 글자가 잡히고 있는 글자가 사라진다.**

부호화와 복호화가 같으면 잘 돌아온다

노트북 입력

```
print('고기'.encode('utf-8').decode('utf-8'))  
print('고기'.encode('euc-kr').decode('euc-kr'))
```

실행 결과

```
고기  
고기
```

어느 쪽이든 상관없다

UTF-8이든 euc-kr이든, **같은 부호로 쓰고 같은 부호로 읽으면** 글자는 돌아온다.

부호화와 복호화가 어긋나면 돌아오지 않는다

노트북 입력

```
print('고기'.encode('utf-8').decode('euc-kr', 'ignore'))  
print(ascii('고기'.encode('euc-kr').decode('utf-8', 'replace')))
```

실행 결과

```
怨潑  
'\ufffd\ufffd\ufffd\ufffd'
```

ignore는 읽을 수 없는 바이트를 **조용히 버리고**, replace는 U+FFFD(REPLACEMENT CHARACTER)로 자리만 남긴다. 둘 다 원래 글자는 잃었다.

...‘占쏙옷’을 아십니까?

동요 한 행은 몇 바이트인가

노트북 입력

```
line = open('songs.txt', encoding='utf-8').readline().strip()
print(line)
print(len(line), len(line.encode('utf-8')), len(line.encode('euc-kr')))
```

실행 결과

```
반짝 반짝 작 은 별 아름답 게 비치 네 동쪽 하늘 에서 도 서쪽 하늘 에서 도
44 100 72
```

셈해 보면

한글 28자 + 공백 16개다. UTF-8은 한글을 3바이트로 적으니 $28 \times 3 + 16 = 100$, euc-kr은 2바이트로 적으니 $28 \times 2 + 16 = 72$ 다.

확인 질문

여기까지 정리해 봅시다

- 1 한 바이트로 나타낼 수 있는 값은 몇 가지인가? 현대 한글은 몇 자인가?
- 2 아스키 코드의 여덟째 비트는 원래 무엇에 썼는가?
- 3 B9 CB 한 쌍이 '뭇'도 되고 다른 글자도 될 수 있는가? 무엇을 더 알아야 하는가?
- 4 '뭇'.encode('euc-kr')가 8바이트인 까닭을 '채움 문자'를 써서 설명해 보자.
- 5 화면에 怨湏이 떴다. 부호화와 복호화에 각각 무엇을 썼겠는가?

1 부호화

2 아스키와 확장 아스키

3 한글 인코딩

4 유니코드와 UTF-8

코드 포인트

한글 음절 영역과 자모

UTF-8

여담

5 코퍼스를 다룰 때

6 마무리

표를 하나로 합치면 된다

유니코드 코드 포인트

형식은 U+ + 십육진수. U+0000부터 U+10FFFF까지 네 자리에서 여섯 자리다. 한글 음절 영역은 AC00-D7A3, 가나다순 11,172자다.

	AC0	AC1	AC2	AC3	AC4	AC5	AC6	AC7	AC8	AC9	ACA	ACB	ACC	ACD	ACE	ACF
0	가 AC00	감 AC10	갸 AC20	갯 AC30	갈 AC40	각 AC50	갬 AC60	거 AC70	검 AC80	겐 AC90	갯 ACA0	결 ACB0	격 ACC0	겟 ACD0	고 ACE0	곰 ACF0
1	각 AD01	갑 AC11	갬 AC21	갯 AC31	갸 AC41	갈 AC51	갬 AC61	걱 AC71	집 AC81	갯 AC91	갯 ACA1	겟 ACB1	겟 ACC1	겟 ACD1	곡 ACE1	곱 ACF1
2	갸 AD02	갸 AC12	갸 AC22	갸 AC32	갸 AC42	갸 AC52	갸 AC62	갸 AC72	갸 AC82	갸 AC92	갸 ACA2	갸 ACB2	갸 ACC2	갸 ACD2	갸 ACE2	갸 ACF2
3	갸 AD03	갸 AC13	갸 AC23	갸 AC33	갸 AC43	갸 AC53	갸 AC63	갸 AC73	갸 AC83	갸 AC93	갸 ACA3	갸 ACB3	갸 ACC3	갸 ACD3	갸 ACE3	갸 ACF3

<https://unicode.org/charts/PDF/UAC00.pdf>

세상의 모든 문자를 한 표에 넣으면 “어느 표로 읽을지”를 다룰 일이 없다.

파이썬은 코드 포인트를 그대로 보여 준다

노트북 입력

```
ord('가')  
hex(ord('가'))  
chr(44032)
```

실행 결과

```
44032  
'0xac00'  
'가'
```

노트북 입력

```
from unicodedata import name, lookup  
  
print(name('가'))  
print(name('힐'))  
print(lookup('HANGUL SYLLABLE GA'))
```

실행 결과

```
HANGUL SYLLABLE GA  
HANGUL SYLLABLE HIH  
가
```

6강에서 본 `dtype='<U21'`의 U가 바로 이 유니코드다.

11,172자가 가나다순으로 이어 붙어 있다

노트북 입력

```
print(chr(0xAC00), chr(0xD7A3))  
print(0xD7A3 - 0xAC00 + 1)
```

실행 결과

```
가 힐  
11172
```

통합 완성형과 다른 점

빠진 글자도, 끼어든 글자도 없다. ‘가’에서 ‘힐’까지 **한 칸도 건너뛰지 않고** 가나다순으로 이어진다. 앞 절의 요건 둘을 모두 지켰다.

28과 588

현대 한글 자모

초성(19개) ㄱ ㅋ ㄴ ㄷ ㄹ ㄲ ㄴ ㄷ ㄹ ㅁ ㅂ ㅅ ㅈ ㅊ ㅌ ㅍ ㅎ

중성(21개) ㅏ ㅑ ㅓ ㅕ ㅗ ㅛ ㅜ ㅠ ㅡ ㅚ ㅜ ㅛ ㅜ ㅠ ㅡ ㅚ ㅜ ㅛ ㅜ ㅠ ㅡ ㅚ

종성(28개) ㅇ ㄱ ㅋ ㄴ ㄷ ㄹ ㄲ ㄴ ㄷ ㄹ ㅁ ㅂ ㅅ ㅈ ㅊ ㅌ ㅍ ㅎ

다음 중성과의 거리

```
ord('개') - ord('가') -> 28
ord('월') - ord('울') -> 28
```

다음 초성과의 거리

```
ord('파') - ord('가') -> 588
ord('줄') - ord('울') -> 588
```

종성이 28가지이므로 28이고, $588 = 21 \times 28$ 이다. 표가 산수로 짜여 있다.

나눗셈 두 번으로 자모가 나온다

노트북 입력

[illegible]

```
def decompose(ch):
    code = ord(ch) - 0xAC00
    return CH0[code // 588], JUNG[code % 588 // 28], JONG[code % 28]

print(decompose('값'), decompose('뵤'), decompose('가'))
```

실행 결과

('ר', ' 'פ', 'אג') ('ט', 'ח', 'ד') ('ר', ' 'פ', ' 'א')

거꾸로 하면 글자가 만들어진다

노트북 입력

```
def compose(cho, jung, jong):  
    return chr(0xAC00  
        + CHO.index(cho) * 588  
        + JUNG.index(jung) * 28  
        + JONG.index(jong))  
  
print(compose('ㅎ', 'ㅣ', 'ㄱ'))
```

실행 결과

획

통합 완성형으로는 못 하던 일

자모순으로 배열되어 있으므로
산수로 글자를 만들 수 있다.

초성 검색, 자모 단위 오타 교정, 음운 규칙 구현이 모두 이 두 함수에서 시작한다.

[가-힣] 이 한글을 잡아내는 까닭

노트북 입력

```
import re  
  
print(re.findall(r'[가-힣]+', 'Hello, 안녕하세요! 2026년 봄'))
```

실행 결과

```
['안녕하세요', '년', '봄']
```

범위 표기가 뜻을 가지려면

(1) 음절 영역이 **끊기지 않고** 이어져 있을 것 (2) 그 순서가 **가나다순**일 것

3강과 8강에서 [A-Z]가 영어에 대해 하던 일을, 한글에 대해서는 [가-힣]이 한다.

UTF-8은 길이가 변하는 부호다

UTF-8 인코딩

코드 포인트	비트 수	N 바이트 표현 ($N \in \{1, 2, 3, 4\}$)
0000–007F	7비트	0xxxxxxx
0080–07FF	11비트	110xxxxx 10xxxxxx
0800–FFFF	16비트	1110xxxx 10xxxxxx 10xxxxxx
10000–10FFFF	21비트	11110xxx 10xxxxxx 10xxxxxx 10xxxxxx

The Unicode Standard, ch. 3, Table 3-6. <https://www.unicode.org/versions/latest/ch03.pdf>

- $007F_{(16)} = 1111111_{(2)} = 127_{(10)}$
- 아스키 문자는 아스키 코드 그대로 1바이트다 (패리티 비트 없이).
- 한글 음절 영역(AC00-D7A3)은 3바이트다.

‘가’를 손으로 부호화한다

3바이트 형식

1110xxxx 10xxxxxx 10xxxxxx

빈 자리 x가 $4 + 6 + 6 = 16$ 개다. 코드 포인트 십육진수 네 자리가 꼭 들어맞는다.

예시: 한글 음절 문자 ‘가’의 UTF-8 인코딩

1 ‘가’의 코드 포인트는 AC00

- $AC00_{(16)} = 1010\ 1100\ 0000\ 0000_{(2)}$

2 열여섯 자리를 4-6-6으로 끊어 형식에 채운다

- 1110**1010** 10**110000** 10**000000**

	AC0	AC1	AC2
0	가 AC00	감 AC10	갸 AC20
1	각 AC01	갑 AC11	갯 AC21
2	깁 AC02	깁 AC12	깁 AC22

<https://unicode.org/charts/PDF/UAC00.pdf>

파이썬으로 확인한다

노트북 입력

```
bytes_ga = '가'.encode('utf-8')
print(bytes_ga)
print(' '.join(f'{b:08b}' for b in bytes_ga))
print(bytes([0b11101010, 0b10110000, 0b10000000]).decode('utf-8'))
```

실행 결과

```
b'\xea\xba\x80'
11101010 10110000 10000000
가
```

3강의 약속을 여기서 지킨다

‘가’는 1글자면서 3바이트다!

‘바람’의 길이는 2이면서 6이다

노트북 입력

```
w = '바람'
print(w.encode('utf-8'))
print(len(w), len(w.encode('utf-8')))
```

실행 결과

```
b'\xeb\xba\x94\xeb\x9e\x8c'
2 6
```

str 글자의 줄. len은 **글자**를 센다. wc -m

bytes 바이트의 줄. len은 **바이트**를 센다. wc -c

3강의 두 수 367,682와 367,377이 갈라진 것도 이 때문이다.

같은 글을 인코딩마다 몇 바이트로 적는가

노트북 입력

```
s = 'Hello 안녕'
for e in ['utf-8', 'utf-16',
          'utf-32', 'euc-kr']:
    print(e, len(s.encode(e)))
```

실행 결과

```
utf-8 12
utf-16 18
utf-32 36
euc-kr 10
```

읽는 법

'Hello 안녕'은 8글자다. UTF-8은 영문자 6개를 1바이트씩, 한글 2자를 3바이트씩 써서 $6 + 6 = 12$ 다. UTF-32는 모든 글자를 4바이트로 적어 $8 \times 4 = 32$ 에 머리표 4바이트를 더한다. euc-kr이 가장 짧지만, 그 표에는 ñ도 이모지도 없다.

그런데도 UTF-8이 이겼다

한글만 보면 손해다

동요 한 행이 euc-kr 72바이트, UTF-8 100바이트다. 40퍼센트 가까이 더 쓴다.

그래도 UTF-8인 까닭

- 1 아스키 파일이 그대로 UTF-8 파일이다.
- 2 아무 데서 잘라도 글자 경계를 찾을 수 있다 (이어지는 바이트는 모두 10xxxxxx).
- 3 바이트 순서를 따질 필요가 없다.
- 4 모든 글자가 한 표 안에 있다.

통합 완성형의 ‘똥’은 둘째 바이트가 63이라 영문자 c와 겹쳤다. UTF-8에서는 **첫 바이트와 나머지 바이트의 모양이 달라** 그런 일이 없다.

표의 넷째 줄은 누가 쓰는가

노트북 입력

```
from unicodedata import name

for ch in ['\U0001F600', '\U0002A6A5']:
    print(f'U+{ord(ch):04X}', ch.encode('utf-8'), name(ch))
```

실행 결과

```
U+1F600 b'\xf0\x9f\x98\x80' GRINNING FACE
U+2A6A5 b'\xf0\xaa\x9a\xa5' CJK UNIFIED IDEOGRAPH-2A6A5
```

U+FFFF를 넘는 글자가 4바이트 자리를 쓴다. 이모지 하나는 len으로 1이지만 바이트로는 4다.

표는 지금도 자란다

유니코드에 붙는 글자

새로운 원소가 발견될 때마다 그 이름을 적을 새 한자가 추가된다.

9FEB 气 84.13	𩇛 GCE-118	𩇛 T5-7C54
9FEC 石 112.5	𩇜 GCE-117	𩇜 T4-6E5D

Unicode 11.0에서 추가된 한자.

<https://www.unicode.org/charts/PDF/Unicode-11.0/U110-4E00.pdf>

북한은 열 글자를 넣어 달라고 요청했다.

The 6 characters arranged from 4-72 to 4-77 in KPS 9566-97 are emphasized characters, which are used most frequently in the publications of our country as the characters whose thickness and size are different from characters in Korean character syllable.

위대한 수령 김일성동지께서는 다음과 같이 교시하시였다.

《전민족이 한결같이 조국통일을 지상의 과제로 인정하고있는 이상 사상과 제도의 차이가 통일을 불가능하게 하는 조건으로는 될 수 없습니다.》(《김일성주작전》 제35권, 316페이지)

김정일동지께서는 혁명사적지에 대한 해설을 들으신 다음 옛 모습 그대로 있는 사적건물들을 돌아 보시었다.

<https://www.unicode.org/L2/L2001/01349-N2374-DPRK-AddSymbols.pdf>

ISO/IEC JTC1 SC2 WG2 WG2/N2244

ENCLOSED CJK LETTERS AND MONTHS

(40)	321D	PARENTHEZIZED KOREAN CHARACTER OJEON
(40)	321E	PARENTHEZIZED KOREAN CHARACTER OHU
한글 성 자 모 자	3244	EMPHASIZED KOREAN LETTER KIM INSIDE KIM IL SUNG
	3245	EMPHASIZED KOREAN LETTER IL INSIDE KIM IL SUNG
	3246	EMPHASIZED KOREAN LETTER SUNG INSIDE KIM IL SUNG
	3247	EMPHASIZED KOREAN LETTER KIM INSIDE KIM JONG IL
	3248	EMPHASIZED KOREAN LETTER JONG INSIDE KIM JONG IL
한글 자 모 자	3249	EMPHASIZED KOREAN LETTER IL INSIDE KIM JONG IL
	327C	CIRLED KOREAN CHARACTER CHAMKO
	327D	CIRLED KOREAN CHARACTER JUEUI

북한의 요청 (WG2/N2244)

321D 오전, 321E 오후, 327C 참고, 327D 주의, 그리고 3244-3249의 여섯 자리.

표준화 기구의 답

ISO/IEC JTC1 SC2 WG2 WG2/N2269

- 3. WG2 N2244
 - DPRK's requests: addition of special characters
 - Comments:
 - 1) We would like to get more detailed explanation for each of the letters in addition to character names.
 - 2) For example, we had difficulty figuring out why ".)" and ".>>" are needed.
 - 3) We will fully support adding these special characters provided that these characters are actually needed.

ISO/IEC JTC1/SC2/WG2의 코멘트 (WG2/N2269).

요약하면

“글자 이름마다 더 자세한 설명을 달라. 예컨대 .) 와 .>>가 왜 필요한지 알기 어려웠다. 실제로 필요한 글자라면 전적으로 지지하겠다.”

그래서 무엇이 들어갔는가

요청한 자리	원한 글자	지금 그 자리
321D, 321E	오전, 오후	요청대로 들어갔다
327C, 327D	참고, 주의	요청대로 들어갔다
3244-3249	김일성, 김정일	다른 글자가 들어갔다

unicodedata.name() 으로 유니코드 15.0에서 확인했다. 3244는
지금 CIRCLED IDEOGRAPH QUESTION이다.

행\컬	72	73	74	75	76	77	78	79	80	81
1	↖	↗	↘	↙	↕	↔	↞	↠	↡	↢
2	▽	▼	▷	◁	▶	◀	○	●	◆	▲
3	h	i	j	k	l	m	n	o	p	q
4	김	일	성	김	정	일				
5	и	ч	ш	щ	ъ	ы	ь	э	ю	я
6	VIII	IX	X							i
7	⁸	⁹	½	⅓	⅔	¼	¾	0	1	
8	Pa	kPa	MPa	GPa	ℓ	μℓ	mℓ	dℓ	kℓ	gal

북한의 KPS 9566 인코딩

결과

북한에서 요청한 10글자 중 4글자는 들어갔고 6글자는 들어가지 않았다.

→ 표에 무엇을 넣을지는 기술이 아니라 합의의 문제다.

- 1 부호화
- 2 아스키와 확장 아스키
- 3 한글 인코딩
- 4 유니코드와 UTF-8
- 5 코퍼스를 다룰 때
정규화
파일을 열 때**
- 6 마무리

같은 '한글'인데 길이가 2도 되고 6도 된다

노트북 입력

```
from unicodedata import normalize
a = normalize('NFC', '한글')
b = normalize('NFD', '한글')
print(a, b, a == b)
print(len(a), len(b))
print(len(a.encode('utf-8')), len(b.encode('utf-8')))
```

실행 결과

```
한글 한글 False
2 6
6 18
```

눈으로는 같고, 기계에게는 다르다. 8강의 물음이 글자 층위에서 또 나타났다.

NFD는 자모를 따로 적는다

노트북 입력

```
for ch in normalize('NFD', '한글'):
    print(f'U+{ord(ch):04X}', name(ch))
```

실행 결과

```
U+1112 HANGUL CHOSEONG HIEUH
U+1161 HANGUL JUNGSEONG A
U+11AB HANGUL JONGSEONG NIEUN
U+1100 HANGUL CHOSEONG KIYEOK
U+1173 HANGUL JUNGSEONG EU
U+11AF HANGUL JONGSEONG RIEUL
```

우리가 나눗셈으로 한 일을 유니코드가 표준으로 해 두었다. 다만 이 자모들은 1100-11FF에 있다. AC00 영역이 아니다.

macOS에서 가져온 파일이 안 걸리는 까닭

노트북 입력

```
print(re.findall(r'[가-힣]+', normalize('NFC', '안녕하세요')))  
print(re.findall(r'[가-힣]+', normalize('NFD', '안녕하세요')))
```

실행 결과

```
['안녕하세요']  
[]
```

빈 목록이 나온 까닭과 처방

NFD의 자모는 가-힣(AC00-D7A3) 범위 밖이다. macOS는 파일 이름을 NFD로 적으므로 거기서 받은 자료와 목록이 어긋난다. 읽자마자 `normalize('NFC', s)`로 맞춘다. 한 줄이면 된다.

open()에 무엇을 적는지가 결과를 바꾼다

노트북 입력

```
open('songs.txt', 'w', encoding='utf-8').write('반짝 반짝 작은 별\n')  
  
print(repr(open('songs.txt', encoding='latin-1').read()))  
print(open('songs.txt', encoding='cp949').read())
```

실행 결과

```
'ë°\x98ì§\x9d ë°\x98ì§\x9d ì\x9e\x91 ì\x9d\x80 ë³\x84\n'  
UnicodeDecodeError: 'cp949' codec can't decode byte 0xec  
in position 14: illegal multibyte sequence
```

오류가 난 쪽이 낫다. latin-1은 아무 바이트나 받으므로 끝까지 조용히 잘못 읽는다. 5강의 결측값과 같다. 티 나는 고장이 낫다.

파일 맨 앞의 보이지 않는 3바이트

노트북 입력

```
open('bom.txt', 'w', encoding='utf-8-sig').write('반짝')
print(open('bom.txt', 'rb').read())

s = open('bom.txt', encoding='utf-8').read()
print(len(s), name(s[0]))
```

실행 결과

```
b'\xef\xbb\xbf\xeb\x0\x98\xec\xa7\x9d'
3 ZERO WIDTH NO-BREAK SPACE
```

두 글자를 썼는데 utf-8로 읽으면 세 글자다. 맨 앞의 폭 0인 글자가 BOM(Byte Order Mark)이고, 엑셀이 CSV에 붙인다. 첫 열 이름이 눈으로는 맞는데 KeyError가 나면 이것을 의심한다. encoding='utf-8-sig'로 읽으면 사라진다.

코퍼스를 다룰 때 적어 둘 네 가지

파일을 열 때

utf-8 기본값으로 삼는다. 새로 만드는 파일은 모두 이것으로 쓴다

utf-8-sig 엑셀이 만든 CSV를 읽을 때

cp949 옛 한국어 자료, 관공서 내려받기 파일

latin-1 오류 없이 읽히므로 **진단용으로만** 쓴다

환경마다 기본값이 다르다

`open()`에 `encoding=`을 적지 않으면 운영체제 설정을 따른다. 윈도우에서는 cp949가 되기도 한다. **언제나 적는다.** 그래야 남의 컴퓨터에서도 같은 결과가 나온다.

같은 이유로 터미널에서도 `wc -m`은 로케일이 UTF-8일 때에만 문자를 센다.

1 부호화

2 아스키와 확장 아스키

3 한글 인코딩

4 유니코드와 UTF-8

5 코퍼스를 다룰 때

6 **마무리**
요약
다음 시간
과제

오늘 풀린 떡밥

그때 본 것	어느 강의	오늘의 이름
WC -c → 367,682	3강	바이트 수
WC -m → 367,377	3강	문자 수. 차 305는 아스키 밖 글자
‘가’는 1글자, 3바이트	3강의 약속	AC00 → EA B0 80
ENTRÉE → ENTRE	4강	É는 ISO 8859-1의 C9
dtype='<U21'	6강	U는 유니코드
동요 한 행	8강	44자 / UTF-8 100바이트 / euc-kr 72바이트
‘바람’의 길이	8강의 물음	str은 2, bytes는 6

한 문장으로

그동안 센 것이 **사람이 읽는 글자**였는지 **기계가 세는 바이트**였는지를 이제는 구별해서 적을 수 있다.

오늘 배운 것

부호화

- 부호화·복호화·부호·코덱
- 한 바이트는 256가지
- 아스키는 7비트, 남은 비트는 패리티

유니코드와 UTF-8

- U+AC00-U+D7A3이 이어진다
- 588과 28로 자모를 쪼갬다
- 한글은 3바이트, 이모지는 4바이트

한글 인코딩

- 11,172자에는 2바이트가 필요하다
- 완성형은 2,350자뿐, '의'는 8바이트
- 통합 완성형은 가나다순을 잃었다

코퍼스를 다룰 때

- str와 bytes는 다른 것
- NFC는 2자, NFD는 6자
- open()에 encoding=을 적는다

확인 질문

오늘 것을 정리해 봅시다

- 1 UTF-8에서 한글 '가'는 몇 바이트인가? 그 3바이트는 어떻게 나오는가?
- 2 `ord('힉') - ord('가')`는 11,145다. 588과 28로 나누어 초·중·종성을 말해 보자.
- 3 [가-힉] 이 한글 한 글자씩을 잡아내려면 어떤 조건 둘이 필요한가?
- 4 눈에 같아 보이는 두 '한글'의 `len`이 2와 6으로 갈렸다. 각각 어떤 정규화 형식인가?
- 5 UTF-8 파일을 `encoding='latin-1'`로 열면 왜 오류가 나지 않는가? 그것이 왜 더 위험한가?

다음 시간에 배울 것 (10월 7일 수요일)

제10강 NSMC 데이터 활용

8강은 “무엇을 하나로 셀 것인가”를 물었고, 오늘은 “글자 하나가 무엇인가”를 물었다. 다음 시간은 마지막 물음이다. **한국어에서 단어가 무엇인가**. Kiwi로 형태소를 나누고 단어구름을 그린다.

그때 오늘 것을 쓴다

영화평을 읽어 들이는 첫 두 줄이 `encoding='utf-8'`과 `normalize('NFC', ...)`다. 그다음에야 형태소 분석기가 돈다.

10월 5일 월요일은 개천절 대체공휴일이므로 수업이 없다.

과제 안내

[숙제04] seaborn 연습

마감 **2026-10-02 12:00 (금요일 정오)**. 그림마다 제목과 축 이름을 붙인다. 제출 전에 [Restart] 후 [Run All]. 남의 컴퓨터에서도 같은 결과가 나와야 한다.

[숙제05] 문자 인코딩 연습

마감 **2026-10-08 12:00 (목요일 정오)**. 노트북이 아니라 **서면 풀이 PDF**를 eTL에 낸다.

- 1 현대 한글이 11,172자인 까닭을 설명한다.
- 2 ‘ㄱ’을 초성으로 가지는 음절이 몇 개인지 설명한다.
- 3 `ord('설') - ord('가')`가 5412인 과정을 설명한다.
- 4 666에서 초성 ‘ㄱ’, 중성 ‘ㅏ’, 종성 ‘ㅊ’을 얻는 과정을 설명한다.

같은 날 마감인 인증 과제 둘

[숙제06] 언어데이터 관리 사례 발표 논문 선정

마감 **2026-10-08 17:00 (목요일 오후 5시)**. 부교재 *The Open Handbook of Linguistic Data Management*의 Part II(14-56장)에서 발표할 장을 1·2·3순위로 골라 장 번호를 낸다. 겹치면 **먼저 낸 사람이** 가져간다.

[숙제07] 모두의 말뭉치 가입 인증

마감 **2026-10-08 17:00 (목요일 오후 5시)**. 국립국어원 모두의 말뭉치에 가입해 **온라인 대화 말뭉치 2021**을 신청하고, [말뭉치 신청 내역] 화면을 hw07-**학번**.png로 eTL에 낸다. 신청서 작성 요령은 문제지 [부록]에 있다.

둘 다 **오늘 해 두는 편이 낫다**. 승인은 담당자가 직접 하고, 11강에서 열 것이 바로 [숙제07]에서 신청하는 그 말뭉치다.